Build brief — Open Sentry Beacon

For an agent building the live application in one session. Assumes no memory of any prior conversation. Read this whole file before writing code; the traps section exists because each trap has already cost somebody a day.

---

1. What this app is

A church pairs **one member** with **one person exploring faith**, and walks with them through six stages. That is the entire product. Everything else — lessons, the library, prayer, meetings — supports that one relationship.

Four roles, and the words matter because they are on every screen:

Role	DB value	Who
Explorer	`ds`	Someone exploring the world of SDA values, at their own pace
Guide	`dm`	The church member walking with them
Director	`admin`	Leads a church's Hope Beacon ministry; usually the pastor
Executive Director	`executive`	The same job across more than one church

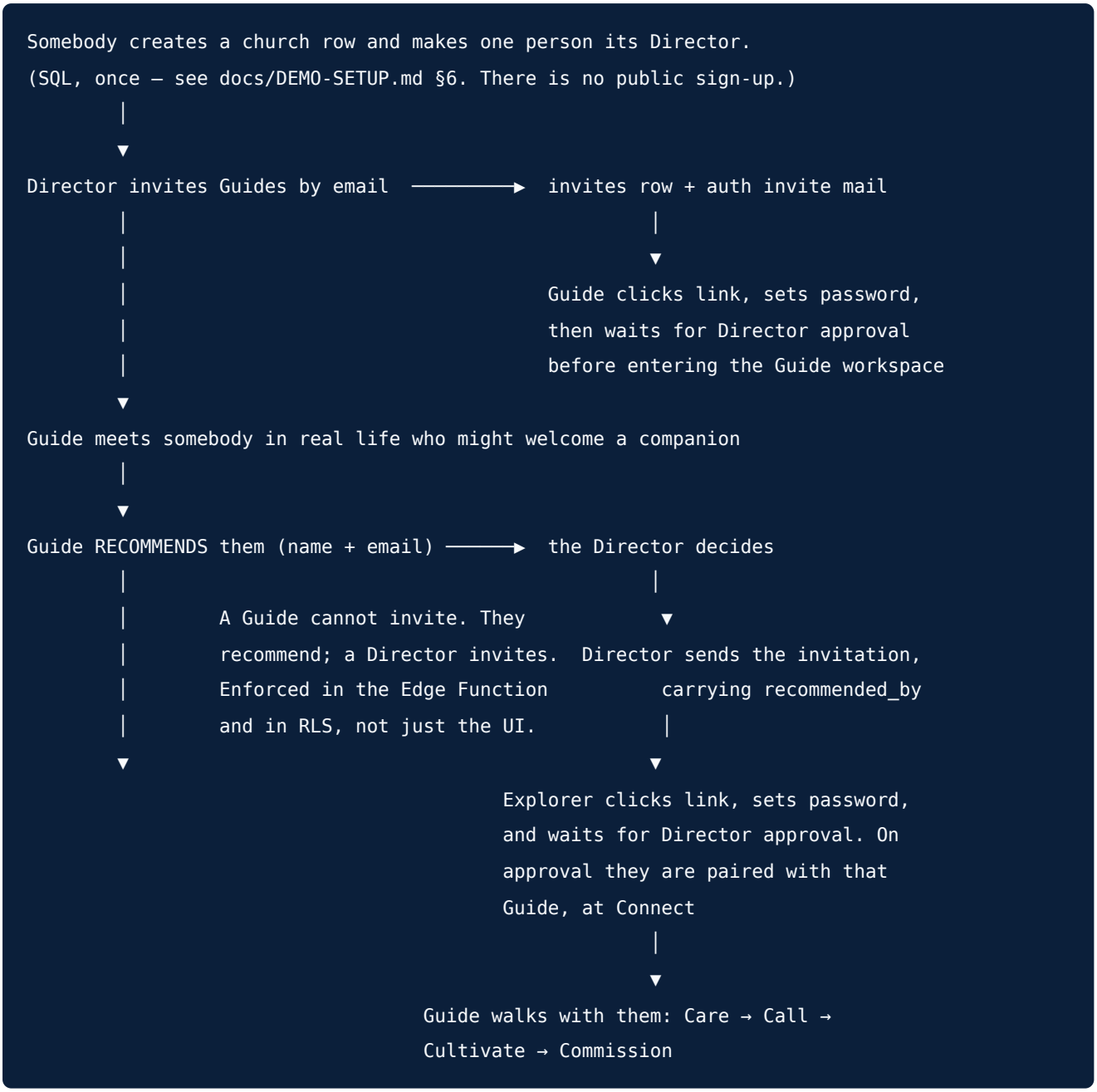
The database stores the old short values (`ds` , `dm` , `admin` , `executive`). Every policy is written against those. **Do not rename them** — the display names live in `lib/brand.ts` and that is the only mapping.

The six stages, in order: `create → connect → care → call → cultivate → commission` .

---

2. The flow of the whole system

Getting a church started



An Explorer is never at Create. By the time they have an account, a Guide already brought them. Create is the Guide's own starting state.

Who sees what

	Sees
Explorer	Their own page, their Guide, what has been shared with them. **Never their own stage.**
Guide	Only the Explorers paired with them. Their conversations, journeys, private notes. Nothing of another Guide's.
Director	Everything in their church: approvals, pairings, the library, aggregate numbers. **Never a conversation.**
Executive Director	The same, across every church they oversee.
Church board	Has no account and never will. The Director reads them numbers.

Two promises to hold on to, because churches ask about both and because both are enforced in the database rather than the UI:

- **Conversations are readable by exactly two people.** One policy, no leadership branch.
- **An Explorer is never shown a journey stage, including their own.** A stage is a note the church keeps to remember where a conversation got to. Shown to the person, it reads as a grade.

---

3. Where the code is now

Done and on `main`:

- Whole app working in **sample-data mode** — library, lessons, lesson series, study shelf, prayer, meetings, materials, analytics, tutorial. This is a complete, demoable product with no backend.
- `lib/brand.ts` — role names, and `roleLabel(role, viewer)`.
- `lib/mode.ts` — `IS_LIVE` / `IS_DEMO`, decided by whether keys exist.
- `lib/supabase/client.ts` — `supabase()`, returns **null** when unconfigured.
- `lib/live/data.ts` — the live data layer for the core loop. Complete.
- `supabase/migrations/0001_core_schema.sql` — schema, helpers, RLS, applied.
- `supabase/migrations/0002_invitations.sql` — invites + redemption, applied.
- `supabase/migrations/0003_invite_approval_gate.sql` — approval boundary and role hardening.
- `supabase/migrations/0004_live_api_permissions.sql` — explicit Data API grants and message realtime.
- `supabase/functions/invite/index.ts` — invitation mail function.
- Live mode's home, email/password sign-in, invitation join, Director, Guide, Explorer, pairing, stage and conversation screens.

No longer true, and kept here only so the change is legible: this section once ended "Lessons, meetings, prayer and materials have no live schema yet." All four have had one for some time, along with guilds and their room, safeguarding reports, hearings, the library, the Guides' room and the conversation's own edit and delete history. The

live half is the app now; the sample deployment is the tutorial, not a holding pen for unfinished features.

---

4. Build order

Ship each step working before starting the next. **Commit after every step** — this environment has lost uncommitted work repeatedly.

Step 1 — Auth and the shell

Make sign-in real when `IS_LIVE`.

- A session provider: read `supabase().auth.getSession()`, subscribe to `onAuthStateChange`, expose `{ session, profile, loading }`.
- `app/login/page.tsx` already branches for demo. Add the live branch: email + password → `signIn()` from `lib/live/data.ts`.
- Route by role after sign-in: `executive` / `admin` → `/admin`, `dm` → `/dm`, `ds` → `/ds`.
- Unapproved accounts see "your account is being reviewed" and nothing else.

Done when: you can sign in as the Director you created in SQL and land on

Step 2 — The Director's screen

- Member list from `listMembers()`.
- Approvals: pending profiles → `approveMember(id, role)`.
- Pairing: pick a Guide and an Explorer → `createPairing()`.
- Invitations: the form calls the Edge Function (step 4).

Done when: a Director can approve somebody and pair two people, and it survives a refresh.

Step 3 — The Guide and the Explorer

- Guide: `listPairings()` filtered to their own, each opening a conversation.
- Conversation: `listMessages()`, `sendMessage()`, `subscribeToMessages()`, `markRead()`.
- Stage: `advanceStage()` — writes the pairing and a `journey_events` row.
- Explorer: `getMyPairing()` — **no stage in the response, by design.**

Done when: two browsers, two accounts, one conversation, messages appearing live.

Step 4 — Invitations

```
const { data, error } = await supabase()!.functions.invoke('invite', {
  body: { email, role, full_name, recommended_by },
});
if (error) setError(error.message);   // show it verbatim
```

Deploy first: `supabase functions deploy invite --project-ref bcpuushjwcejytdthlnn`

Done when: a real invitation email arrives, the link sets a password on an unapproved account in the right role, and a Director's separate approval opens that role's workspace.

Step 5 — The rest of the schema

Done. Lessons, meetings, prayer and the library all shipped, and the numbers this step once guessed at were taken by other work: read

rather than writing them. Everything after `0011` is dated rather than numbered, so two people adding a migration on the same day do not collide.

The instruction that has not changed is the one that mattered: copy the **policies**, not just the tables. A table without its policy is a table the Data API will hand to anybody who asks.

---

5. Patterns to follow

Branch on mode at the component boundary, not inside every function.

```
import { IS_LIVE } from '@/lib/mode';
import * as live from '@/lib/live/data';
import { useDemo } from '@/lib/demo/store';

// Read both, use one. Hooks cannot be called conditionally.
const demo = useDemo();
const [rows, setRows] = useState<Thing[] | null>(null);

useEffect(() => {
  if (!IS_LIVE) { setRows(demo.things); return; }
  let alive = true;
  live.listThings().then((r) => alive && setRows(r)).catch(() => alive && setRows([]));
  return () => { alive = false; };
}, [IS_LIVE, demo.things]);
```

Every async load needs the `alive` guard. Without it, navigating away mid-request sets state on an unmounted component and React logs a warning that buries real errors.

Never `select('*')` for an Explorer's own data. Name the columns. See

Errors are shown, not swallowed. `catch { }` that sets an empty array is fine for a dashboard line; it is wrong for a save. If a write fails, the person must be told.

---

6. Rules that must not be broken

Each of these has already been broken once in this project.

1. **RLS stays on, on every table.** The anon key is public — it ships to every browser by design. The policies are the only thing between it and the data. New table → `alter table ... enable row level security` in the same migration, always.

1. **A policy decides WHICH ROWS; a grant decides WHICH COLUMNS.** RLS cannot express "only this column". `messages_mark` existed so a recipient could stamp `read_at`, and because a policy says nothing about columns it also let either person in a pairing rewrite the other's words. An audit for the same shape found three more, including a Guide able to rewrite their Explorer's prayer request and publish a private one to the whole church. Before writing an UPDATE policy, ask what the app actually writes, then `revoke update` and `grant update (that_column)`.

1. **Role is not church.** `is_admin()` and `is_executive()` check the caller's role and nothing else; `leads_church(c)` and `manages_church(c)` check the role and that church. Anything taking a member id wants the second. Both guardian-consent functions used the first, so a Director of one church could alter guardian consent for a minor in another.

1. `service_role` **never leaves the server.** Not in this repo, not in any `NEXT_PUBLIC_*`, not in a browser file. It bypasses every policy. Only the Edge Function holds it.

1. **A write that wakes every screen needs a ceiling, and a watcher needs a debounce.** One insert becomes one recount per open app. `hold_the_pace` caps a signed-in account at 40 writes a minute across the conversation, the Guild wall and the Guides' room; on the client, watch tables through `useKeepUp` and never a raw realtime channel, which has no settling and turns a burst of forty messages into forty recounts per viewer.

1. **Nothing trusts client metadata for a privilege.** `signUp()` lets any caller attach arbitrary `data`. A trigger reading `role` from it hands Executive Director to the internet. Role, church and approval come from the `invites` table. See the long comment in `0002_invitations.sql`.

1. **No** `setMyRole()` **in the live layer.** The demo store has one; it is a toy there and a privilege escalation here.

1. **Cross-table policy references go through a** `SECURITY DEFINER` **helper.** A direct subquery between `profiles` and `pairings` causes infinite recursion and every affected read fails outright. Use `church_of()` and `is_paired_with()`. This shipped once and broke the whole app.

1. **A conversation is readable by two people.** No Director branch, no executive branch, no audit exception. Adding one changes what this app is.

1. **JavaScript never enforces access.** The browser can call PostgREST directly with the same key. A filter in `lib/live/data.ts` is for correctness or for fewer rows — never for security. If you catch yourself writing "and only if the user is an admin" in TypeScript, the rule belongs in a policy.

---

7. Traps that have already cost time here

- **A zero proves nothing.** Testing "can an outsider read this?" and getting 0 is meaningless until you have shown the row exists and that somebody entitled reads 1. Every security test needs a positive control in the same transaction. Three separate "airtight isolation" results here were actually null users and empty tables.
- **An empty result is not a finding until the query is proved to work.** The sibling of the zero above, and it cost time twice in two days. A log query filtered on `status_code` instead of `response.status_code` returned nothing and read exactly like a clean system. A probe counting rows in a table it had just written returned zero because it was still acting as a role RLS hides those rows from. And a hand-rolled PDF text scraper reported a phrase absent from a document containing it, because it was reading embedded font programs rather than page text. Before believing an empty answer, make the same query return something you already know is there.
- **Profile triggers silently ignore writes.** `lock_privileged_profile_columns` pins `role` and `church_id` when the caller is not privileged. A test fixture that promotes somebody with plain SQL and no `auth.uid()` does nothing at all and reports success. Use `alter table public.profiles disable trigger user` inside a rolled-back transaction for fixtures.
- **A comment claiming an invariant is not an invariant.** A test here said it derived role names from the brand map while the line below spelled them out; it broke on the next rename and nobody noticed for two renames.
- `markitdown`**-style greps miss wrapped JSX.** Retired vocabulary hid across line breaks through two deliberate sweeps. `tests/brand-consistency.mjs` now catches it — run it.

---

8. Verifying

This repo is public, so GitHub Actions cost nothing here. Locally:

```
npm run verify:all                # everything
node tests/no-backend.js          # no keys committed; still runs unconfigured
node tests/brand-consistency.mjs  # retired vocabulary has not crept back
npx tsc --noEmit                  # types
npm run build                     # must pass before any push
```

After any policy change, re-run the attack suite documented at the top of

controls. It is the cheapest test in the project and the only one that checks the promise the README makes.

---

9. The fallback, and why it exists

Unset `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` in Vercel and redeploy: the app falls back to sample data and every feature works.

Keep that deployment alive. It is the demo that cannot fail, and `lib/mode.ts` asks "do I have keys?" rather than reading a flag precisely so the fallback is one setting rather than a code change somebody has to remember.